Intellego: A Local-First AI Software Engineering Platform with Semantic Repository Graphs and Diff-Gated Code Editing

[Author Name 1], [Author Name 2], and [Author Name 3]
[Department], [Institution]
[City, Country] — [email address]

***Abstract*—**Large language model (LLM) coding assistants are increasingly used for repository-scale work, yet most are delivered as cloud-connected tools that transmit code to remote services, apply edits with limited review, and reason about a repository mainly through retrieved text fragments. This paper presents Intellego, a native desktop platform that combines an Electron and React interface with a Python FastAPI sidecar bound to the loopback interface, a per-workspace SQLite store, and NVIDIA Nemotron as its sole language model. The system builds a semantic knowledge graph of repository symbols using Python's abstract syntax tree and pattern-based extraction for JavaScript and TypeScript, streams model responses over WebSocket with an HTTP fallback, and converts model-proposed changes into unified-diff previews that reach the disk only after explicit developer approval through a single-use, time-limited identifier. We describe the architecture, the edit-gating protocol, and the guardrails that confine file access to the opened workspace. Verification comprises 16 automated engine tests, 6 service-level frontend tests, and a production build of 52 modules without errors. We also report candidly the gap between the specified design and the shipped code: Tree-sitter parsing, multi-agent orchestration, and incremental re-indexing remain future work. The results show that approval-gated editing and local persistence can be delivered with a small, auditable code base and define a concrete path toward quantitative evaluation.

***Keywords*—**software engineering assistants, large language models, semantic code graph, human-in-the-loop editing, local-first architecture

1. INTRODUCTION

Large language models trained on source code have moved from completion engines toward assistants that read, explain, and modify entire repositories [1], [2]. Benchmarks such as SWE-bench illustrate both the promise and the difficulty: resolving a real GitHub issue requires locating the relevant code across many files, not merely generating a function [3]. At the same time, commercial assistants are commonly delivered as cloud-connected editors or command-line agents. This raises three practical concerns for developers and organisations: where source code is transmitted and stored, how much of the repository's structure the tool actually understands beyond retrieved text, and what prevents a mistaken or manipulated model output from modifying files without review.

Intellego is a native desktop application that addresses these concerns through a deliberately small architecture. A user opens a local folder as a workspace; a Python engine scans it into a semantic graph of symbols, persists that graph and the conversation history in a workspace-local SQLite file, and answers questions through a streamed call to a single hosted model. When the model proposes a change, the change is never written directly. It is turned into a unified-diff preview with a single-use identifier, shown in a diff viewer, and applied only when the developer accepts it.

The contributions of this paper are fourfold: (i) a loopback-only sidecar architecture that keeps indexing, persistence, and file mutation on the developer's machine, with the model endpoint as the only external call; (ii) a symbol-level semantic graph builder with a uniform schema across Python, JavaScript, TypeScript, and generic files; (iii) an edit-gating protocol that couples a proposal format, a preview store with expiry, and workspace-root path validation; and (iv) an honest audit of specified versus implemented functionality, together with the verification evidence that exists today. The remainder of the paper reviews related work (Section 2), presents the methodology (Section 3), describes the implementation (Section 4), discusses results (Section 5), considers ethical and legal aspects (Section 6), and concludes (Section 7).

2. REVIEW OF LITERATURE

2.1 Literature

Research on machine learning for source code treats software as a natural-language-like artefact with statistical regularities, a view surveyed comprehensively in [4]. Pre-trained encoders such as CodeBERT aligned natural language with programming languages for search and documentation tasks [5], while decoder models such as Code Llama specialised general language models for infilling and long-context code generation [6]. The model used in Intellego belongs to NVIDIA's Nemotron family, which was released with an emphasis on synthetic-data alignment and is served through an OpenAI-compatible interface [7].

Because repositories exceed a model's context window, assistants must decide which code to show. Retrieval-augmented generation conditions a generator on retrieved passages [8], but long contexts are not used uniformly: information placed in the middle of a long prompt is recalled less reliably than information at the beginning or end [9]. This

motivates structured, symbol-level selection over bulk text retrieval.

Agentic approaches interleave reasoning with tool use. ReAct formalised the pattern of alternating thoughts and actions [10], and SWE-agent showed that a carefully designed agent-computer interface, rather than the base model alone, strongly influences success on repository tasks [11]. Structural code analysis commonly relies on incremental parsers such as Tree-sitter, which produce concrete syntax trees that tolerate errors and update efficiently [12]. For presenting changes, line-based differencing in the tradition of Hunt and McIlroy remains the standard review format [13]. On the safety side, path traversal is a long-standing class of vulnerability in which crafted file names escape an intended directory [14], and it becomes newly relevant when file names are produced by a language model.

Field evidence on assistants is encouraging but conditional. Controlled experiments with a commercial completion tool report faster task completion for developers who use it [2], yet suggested code still has to be verified by the person who accepts it. How easily a suggested change can be inspected therefore shapes whether assistance translates into correct software, which argues for presenting edits as reviewable diffs rather than as opaque writes.

Two further observations from the literature inform the design. First, graph-structured representations of code, such as call and dependency graphs, give an assistant explicit relations that a flat list of retrieved snippets lacks, and they can be presented to humans for inspection. Second, the more capability an agent is given, the more its action space must be constrained; interface design, not model scale alone, determines how safely an agent operates on a file system. Intellego applies both observations at small scale.

2.2 Research Gaps

Four gaps motivate this work. First, many assistants rely on text retrieval and do not expose an inspectable symbol-level model of the repository. Second, review of model-written edits is often optional or coarse-grained, and the guarantees that a rejected edit never touches disk are rarely stated as an explicit protocol. Third, code and conversation data commonly leave the machine through hosted indexing and storage. Fourth, published descriptions of assistant platforms seldom distinguish between the architecture that is specified and the functionality that is actually implemented, which makes comparison difficult.

2.3 How This Paper Addresses the Gaps

Intellego stores an explicit symbol graph that the user can browse; it separates proposal from application through a preview identifier with a one-hour lifetime; it persists everything except the model call locally; and this paper reports implemented and planned features in separate columns (Table 3) rather than blending them.

3. PROPOSED METHODOLOGY

3.1 Design Principles

Four principles guide the design. *Local first*: the engine binds only to 127.0.0.1 and the workspace database lives inside the opened folder. *Human approval for mutation*: no model output writes to disk without an explicit accept action. *Uniform response contract*: every endpoint returns an envelope with a success flag, a message, an optional data payload, and an optional error object. *Honest degradation*: if streaming over WebSocket fails, the client falls back to a chunked HTTP request, and if the model key is absent the engine reports a structured error rather than a silent failure.

3.2 Semantic Knowledge Graph

The Repository Intelligence Engine walks the workspace, skips configuration-defined ignore directories, and emits one node per symbol. Formally, the graph is

$$G = (V, E),\ E \subseteq V \times V \tag{1}$$

where each vertex $v \in V$ is a tuple (name, kind, file, line) with kind $\in$ {class, function, variable, import, export}, and an edge $(v_i, v_j) \in E$ records that v_i depends on v_j. Python files are parsed with the standard abstract-syntax-tree module, which yields exact definitions and imports. JavaScript and TypeScript files are processed with regular-expression patterns for declarations, imports, and exports, which is faster to implement but cannot resolve scope. Other file types are registered as generic file-level nodes so that they remain visible in the graph.

3.3 Agent Runtime and Proposal Protocol

The agent is a single streamed call to the language model. A system prompt frames the assistant's role, injects workspace metadata and selected symbols, and instructs the model to express any file change as a tagged block of the form <proposed_change file="…"> containing the complete new content. The engine parses these blocks from the completed stream, and emits a marker line carrying a JSON preview payload so that the client can render diffs inline with the message text. Roles such as planner or reviewer are expressed only at the prompt level; no separate agent processes exist in the current release.

3.4 Diff-Gated Edit Protocol

For each proposal, the tool runtime first checks that the target path stays inside the workspace root r:

$$\rho(f, r) = 1 \ \text{iff}\ \text{resolve}(r/f) \sqsubseteq r \tag{2}$$

where resolve canonicalises the path and $\sqsubseteq$ denotes "is contained in". Absolute paths, parent-directory sequences, and directory targets are rejected. If $\rho = 1$, the runtime computes a unified diff against the current file, stores the proposal under a fresh UUID p with creation time t_p, and returns the preview. An apply request is honoured only if

$$A(p, t) = 1 \ \text{iff}\ p \in S \ \wedge\ t - t_p \le 3600\ s \tag{3}$$

where S is the set of live preview identifiers. A successful apply writes the file and removes p from S, so each preview is single-use; an unknown or expired identifier yields a PREVIEW_EXPIRED error. Rejecting a preview simply

discards it. Version-control operations follow the same philosophy: the git endpoints report status and create commits only when the user presses the corresponding control.

3.5 Persistence

Workspaces, chat sessions, messages, and symbols are stored in a SQLite database at .intellego/workspace.db inside the workspace. The renderer additionally keeps projects, chat lists, and preview records in browser local storage so that the interface restores instantly on launch.

3.6 Streaming Transport

Responses from the model are long and arrive incrementally, so the client opens a WebSocket to the /chat/stream endpoint and renders text as tokens arrive. The engine forwards each chunk, accumulates the full text, and after the final chunk runs the proposal parser. If the socket cannot be established, the client issues a chunked HTTP POST to a sibling endpoint and reads the response body as a stream. Both paths share the same agent code and the same envelope for errors, so the user interface contains a single rendering routine and the transport is an implementation detail.

3.7 Threat Model and Guardrails

The threat model assumes a single, trusted developer on a personal machine, an untrusted model output, and possibly untrusted repository content. The principal risks are therefore unintended writes, writes outside the workspace, replay of stale previews, and accidental publication of changes. Table 4 lists the guardrails that address each risk and the point at which each is enforced.

Table 4: Guardrails and where they are enforced.

Guardrail	Risk addressed	Enforced in
Workspace-root scoping	Writes outside the project	Tool runtime
Path and directory rejection	Traversal, directory targets	Tool runtime
UUID, single use	Replay of an applied edit	Preview store
One-hour expiry	Stale diffs against changed files	Preview store
Loopback binding	Network exposure of the engine	Engine start-up
Explicit user click	Unreviewed edit or commit	Diff viewer, git panel

The guardrails are complementary. Path validation limits where a write can land, the preview store limits when it can occur, and the user interface limits who can trigger it. Prompt injection from repository text is not eliminated by any of them, but its consequences are bounded because it can at most produce a proposal that the developer must still accept.

3.8 Context Budget

Every prompt must fit within the model's context window C. Conceptually, the tokens consumed by the system prompt, the selected symbols, the conversation history, and the user request must satisfy

$$T_sys + T_sym + T_hist + T_usr \leq C \quad (4)$$

Because information in the middle of a long prompt is used less reliably [9], keeping the symbol and history terms small is preferable to filling the window. The current release applies only simple selection and does not implement a dedicated budgeting component; that role was assigned to the planned Context Engine and is listed as future work. The constraint is nevertheless stated here because it determines how the graph will be consumed once such a component exists.

4. IMPLEMENTATION

4.1 System Architecture

Fig. 1 shows the runtime structure. The Electron main process hosts a React 18 renderer built with Vite and exposes only a narrow preload bridge, mainly for the native folder dialog [15]. The renderer talks to a FastAPI engine [16] over HTTP and WebSocket [17] on the loopback interface. Inside the engine, four cooperating components handle workspace lifecycle, symbol scanning, the streamed agent, and the tool runtime. SQLite provides embedded persistence [18], and the hosted Nemotron endpoint is the only outbound dependency.

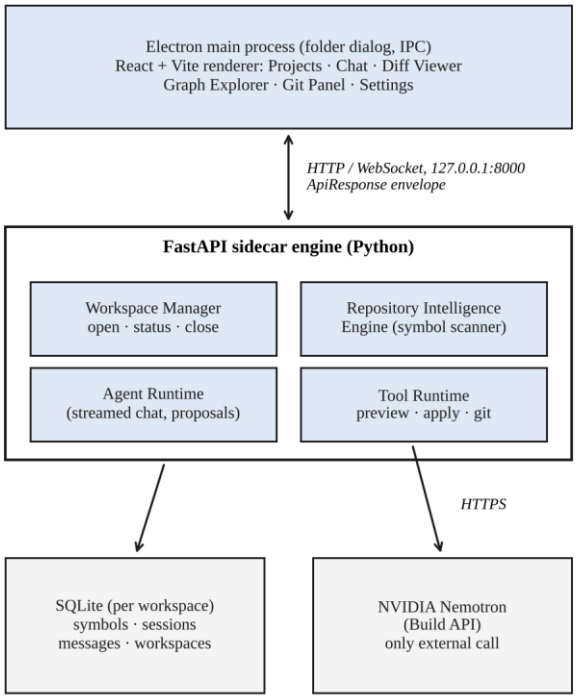


Figure 1: Runtime architecture of Intellego.

4.2 Technology Stack

Table 1: Technology stack.

Layer	Technology	Role
Shell	Electron	Native window, folder dialog, IPC
UI	React 18, Vite 5	Chat, diff viewer, graph explorer, git panel
Engine	Python, FastAPI	REST, WebSocket, scanning, tools
Storage	SQLite	Symbols, sessions, messages
Model	NVIDIA Nemotron	Streamed chat completions
Tests	Python test suite, Node test script	Engine and service checks

4.3 Engine Services

The engine exposes workspace open, status, and close operations; a scan operation that populates the symbol table; chat endpoints, including the /chat/stream WebSocket and a chunked POST fallback; edit preview and apply endpoints; and git status and commit endpoints. All handlers return the common envelope. A command-line demonstration script exercises the same services without the interface, which simplifies debugging.

4.4 Frontend

The renderer provides a project view, a chat mode with a markdown renderer, a workspace panel, a diff viewer that presents proposals before they are applied, a graph explorer for the symbol graph, a git panel, and a settings dialog. Service modules wrap the engine API, the agent runtime, and the tool runtime. Two visual systems coexist in the style sheets: a dark application shell and a paper-and-ink document surface with square corners and offset shadows used for the diff and graph views.

4.5 Edit Flow

Fig. 2 traces one edit from prompt to disk. The only transition that writes a file is the accept branch, and it is guarded by the check in (3).

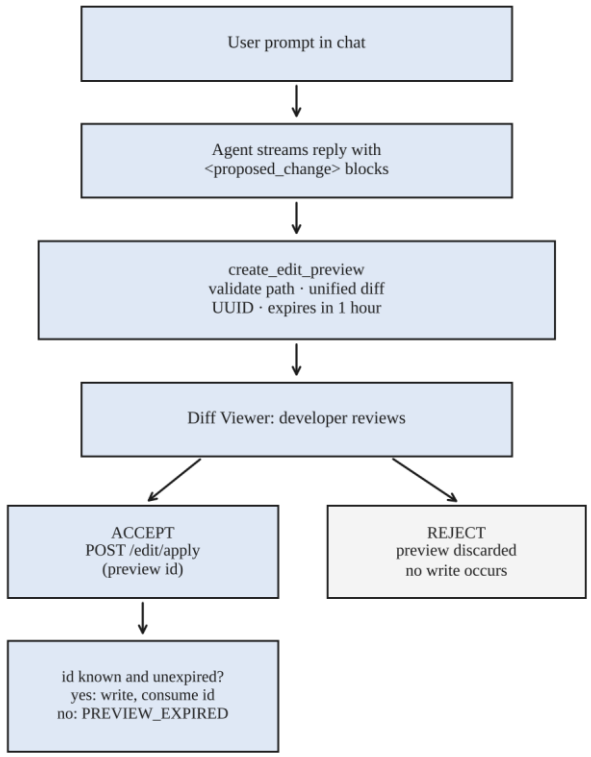


Figure 2: Edit-gating flow from proposal to approved write.

4.6 Data Model

Fig. 3 summarises the persistence schema. A workspace row anchors everything else: each workspace owns many chat sessions, each session owns an ordered list of messages, and the scanner writes symbol rows that reference the workspace. Because the database file lives in the workspace folder, deleting that folder's .intellego directory removes all derived data, and copying the folder to another machine carries the history with it. Symbol rows carry the kind and dependency fields defined in Section 3.2, so the graph explorer can be populated by a single query.

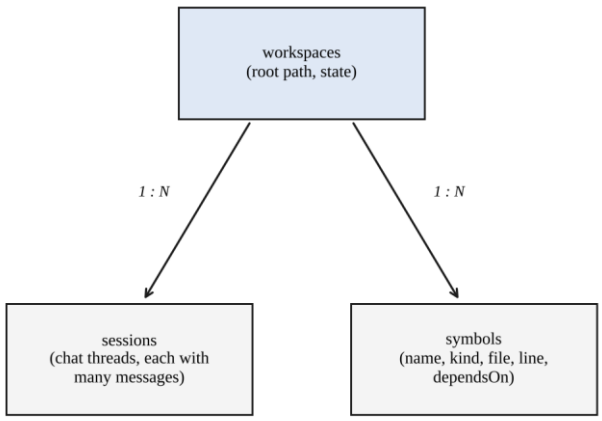


Figure 3: Persistence data model.

4.7 Testing Strategy

Testing is split by layer. The engine test suite contains 16 tests that exercise the scanner, the database layer, and the tool

runtime, including the rejection paths for unsafe file names and expired previews. A separate script of 6 tests exercises the frontend service modules that wrap the engine API. The renderer is additionally checked by a production Vite build, which bundles 52 modules and fails on import or syntax errors. A command-line demonstration script allows manual end-to-end checks without launching the desktop shell.

4.8 Developer Workflow

A typical session proceeds as follows. The developer launches the application and selects a folder, which opens the workspace and triggers a scan. The graph explorer then shows the discovered symbols. In chat mode the developer asks a question or requests a change; the answer streams into the conversation. When the answer contains proposals, the diff viewer lists each affected file. The developer accepts or rejects each proposal, and may then review status in the git panel and create a commit with a message of their choice. Every state-changing step in this workflow therefore corresponds to a visible user action.

4.9 Client-Side State

Besides the engine's database, the renderer keeps lightweight state in browser local storage: the list of projects, the chat list for each project, and the preview records needed to redraw the diff viewer after a reload. This state is a convenience cache rather than a source of truth. If it is cleared, the engine can still return the workspace status and the stored sessions, and previews simply expire. Settings such as the model key are read from the environment by the engine and are never written to local storage by the interface.

4.10 Engineering Notes

Three engineering decisions are worth recording. First, the sidecar is started as a separate process rather than embedded, so that the Python engine can be run, tested, and debugged without the desktop shell. Second, the uniform response envelope allowed the frontend services to share one error-handling routine and made the test scripts short. Third, proposals are expressed as tagged text rather than as a model-specific function-calling format; this keeps the agent independent of any single provider's tool-calling schema and makes the protocol easy to inspect in logs, at the cost of relying on the model to follow the tag format.

5. RESULTS AND DISCUSSION

5.1 Verification Evidence

No controlled user study or latency benchmark has been run yet, so this section reports only the verification evidence that exists. Table 2 summarises it.

Table 2: Verification evidence.

Check	Outcome
Engine test suite	16 tests
Frontend service tests	6 tests
Production build (Vite)	52 modules, 0 errors
Repository graph of this code base	1,291 nodes; 1,264 edges; 141 communities
Guardrail audit	Path, directory, and expiry checks present

The graph statistics come from running a graph-extraction tool over the Intellego repository itself and illustrate the scale of a mid-sized project; they are not a measurement of Intellego's own scanner. The relatively low edge-to-node ratio is expected because file-level and symbol-level nodes dominate and cross-file dependency resolution is limited, as discussed below.

5.2 Specified Versus Implemented

The project's planning documents describe a broader system than the code. Table 3 separates the two.

Table 3: Specified versus implemented.

Component	Specified	Implemented
Parsing	Tree-sitter	Python ast; JS/TS regex
Agents	Multi-agent graph	Single streamed call; prompt roles
Context	Context Engine	Symbols and chat history
Indexing	Incremental, file watcher	Full scan on demand
Edit safety	Diff before apply	Implemented and gated by UUID
Persistence	Local SQLite	Implemented

Reporting this gap serves reproducibility: a reader who expects incremental re-indexing or parser-grade JavaScript analysis would otherwise be misled.

5.3 Error Analysis

Several failure modes follow directly from the design. (a) *Regex extraction* misses nested or dynamically generated declarations in JavaScript and TypeScript and cannot distinguish shadowed names. (b) *Whole-file proposals* make diffs large when the model rewrites unchanged regions, and a truncated stream can produce an incomplete file; the preview step makes this visible but does not prevent it. (c) *Prompt-level roles* give no isolation between planning and editing, so a poorly specified request can yield an unfocused answer. (d) *Expiry* after one hour forces regeneration for slow reviews. (e) *Stale graphs* occur because a full rescan is required after external edits. None of these corrupts the workspace, because mutation remains behind the approval gate.

5.4 Deployment Considerations

Intellego needs Python and Node runtimes and a model key supplied through an environment file. The engine currently allows cross-origin requests from any origin; because it binds to loopback this is low risk, but a restrictive origin list should replace the wildcard before packaging. The repository snapshot examined for this paper also contained a populated environment file that the ignore rules did not exclude, so a credential could be committed by accident; the file should be added to the ignore list and any exposed key rotated. Network use is limited to one HTTPS endpoint, which simplifies firewall policy for organisations.

5.5 Qualitative Discussion

Compared with cloud-integrated editors and command-line agents, Intellego makes different trade-offs rather than claiming superior capability. It keeps indexing and storage on the developer's machine and makes every write reviewable, but it offers a narrower language analysis, a single model, and no autonomous multi-step execution. Mature assistants provide deeper static analysis, more models, and background agents that Intellego does not attempt. Its value lies in being small enough to audit: the entire edit path, from proposal parsing to file write, is a few hundred lines of Python, which makes the safety argument in Section 3.7 checkable by reading code rather than by trusting documentation.

5.6 Threats to Validity

The evidence in this paper is limited in three ways. *Construct validity*: passing tests demonstrate that specified behaviours hold, not that the assistant is useful. *Internal validity*: the tests were written by the same team that wrote the code and may share its blind spots. *External validity*: the system has been exercised on a small number of repositories and a single model, so results may not generalise to large monorepos or other models. Addressing these threats requires an independent evaluation, which is listed as future work.

5.7 Proposed Evaluation Protocol

To turn the qualitative findings into measurements, we propose the protocol in Table 5. The primary behavioural measure is the proportion of proposals that developers accept,

$$\alpha = N_acc / N_prop \quad (5)$$

where N_prop is the number of previews shown and N_acc the number applied. A very low α suggests poor proposals, while a very high α combined with later defects suggests rubber-stamp review, so α should be read together with the defect rate of accepted changes. Scan cost should be reported per thousand lines of code so that results transfer across repositories. None of the values in Table 5 has been measured yet; the table specifies what will be collected.

Table 5: Planned evaluation metrics (not yet measured).

Metric	Definition	Purpose
Scan time	Seconds per KLOC	Indexing cost
First-token latency	Prompt to first chunk	Responsiveness
Acceptance rate α	Accepted / proposed	Proposal quality
Defect rate	Accepted edits later reverted	Review reliability
Task success	Issues resolved on a benchmark	Capability
Review effort	Minutes per accepted diff	Human cost

The protocol would be run on at least three repositories of different sizes and languages, with the same prompts replayed against each configuration so that differences can be attributed to the system rather than to the task.

5.8 Lessons Learned

The most instructive finding is the divergence between documentation and code. Planning documents written early in the project described an ambitious pipeline, and later progress notes sometimes recorded intentions as if they were achievements. An audit that compares each claimed component against the code path that implements it, as in Table 3, is cheap and should be repeated before every release. A second lesson concerns safety: placing the single mutating operation behind one narrow function made the guardrails easy to test and easy to review, whereas safety logic scattered across handlers would have been hard to audit.

5.9 Reproducibility

The repository includes a run guide and a demonstration script so that results can be reproduced. A reader installs the Python requirements and the Node dependencies, supplies a model key through the environment file, starts the engine on the loopback port, runs the engine test suite and the service test script, and finally launches the renderer. Because every test targets deterministic local behaviour, such as path validation, expiry, scanning, and database access, the outcomes in Table 2 do not depend on network access or on the model's output. Model-dependent behaviour, such as the quality of proposals, is by contrast non-deterministic and should be evaluated with repeated runs and fixed prompts as described in Section 5.7.

6. ETHICAL AND LEGAL CONSIDERATIONS

Privacy and data handling. Source code selected for a prompt is sent to a third-party model provider, so developers must ensure that this is permitted by their employer's policy and by the provider's terms. Persisted chats and symbols remain local, but they may themselves contain sensitive excerpts and should be protected with ordinary disk controls.

Accountability and safety. The approval gate keeps a human responsible for every change, which mitigates the risk of silent modification, but it does not guarantee that an approved change is correct. Reviewers should still run tests

and inspect diffs critically, particularly because large diffs encourage cursory approval.

Intellectual property and licensing. Generated code may resemble licensed training data, and the licence status of such output is unsettled in many jurisdictions. Teams should apply their usual licence-compliance checks. The model and each dependency are subject to their own licences and acceptable-use terms.

Secrets and security. Credentials must not be stored in version control. Path-containment checks reduce the risk that a manipulated model output writes outside the workspace, but prompt injection through repository content remains possible and motivates the review step.

Transparency and fairness. Users should be told when text in the interface is machine generated, and the interface should not present proposals as authoritative. Because the model's behaviour reflects its training data, it may perform unevenly across languages and frameworks, and teams relying on it for less common stacks should calibrate their review effort accordingly. Finally, developers remain responsible for compliance with applicable data-protection law when repositories contain personal data.

7. CONCLUSION AND FUTURE WORK

7.1 Key Contributions

This paper presented Intellego, a local-first desktop assistant that couples a symbol-level repository graph with an approval-gated edit protocol. Its main contributions are a loopback-only sidecar architecture, a uniform graph schema across languages, a single-use and time-limited preview mechanism backed by workspace-root validation, and a transparent account of which specified features are present in the code.

7.2 Limitations

The scanner is shallow for non-Python languages, the agent is a single model call, graphs are rebuilt rather than updated incrementally, and the only model is a hosted one. Verification consists of automated functional tests and a build; there is no quantitative measurement of answer quality, latency, or developer productivity, and no comparison with other assistants on a benchmark.

7.3 Future Work

Planned directions include replacing regex extraction with Tree-sitter grammars [12], adding a file watcher for incremental re-indexing, introducing separate planner, coder, and reviewer agents, supporting patch-level rather than whole-file proposals, adding local model back ends, and evaluating the system on a repository-level benchmark such as SWE-bench [3] together with a user study of review effort. A further goal is to make the graph useful beyond chat, for example by highlighting the symbols affected by a pending diff and by estimating the blast radius of a change before it is accepted. Packaging with signed installers, a secure store for credentials in place of an environment file, and an opt-in telemetry-free update channel would complete the path from research prototype to deployable tool.

REFERENCES

[1] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[2] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv:2302.06590, 2023.

[3] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[4] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–37, 2018.

[5] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings Assoc. Comput. Linguistics: EMNLP*, 2020, pp. 1536–1547.

[6] B. Rozière *et al.*, "Code Llama: Open foundation models for code," arXiv:2308.12950, 2023.

[7] B. Adler *et al.*, "Nemotron-4 340B technical report," arXiv:2406.11704, 2024.

[8] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[9] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[10] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.

[11] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[12] "Tree-sitter: An incremental parsing system for programming tools." tree-sitter.github.io. [Online]. Available: https://tree-sitter.github.io/tree-sitter/

[13] J. W. Hunt and M. D. McIlroy, "An algorithm for differential file comparison," Bell Labs, Murray Hill, NJ, USA, Comput. Sci. Tech. Rep. 41, 1976.

[14] "Path traversal," OWASP Foundation. [Online]. Available: https://owasp.org/www-community/attacks/Path_Traversal

[15] "Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS." electronjs.org. [Online]. Available: https://www.electronjs.org/

[16] "FastAPI framework, high performance, easy to learn, fast to code." fastapi.tiangolo.com. [Online]. Available: https://fastapi.tiangolo.com/

[17] I. Fette and A. Melnikov, "The WebSocket protocol," IETF, RFC 6455, Dec. 2011.

[18] "SQLite home page." sqlite.org. [Online]. Available: https://www.sqlite.org/